

**CS570 Spring 2025: Analysis of Algorithms****Exam II**

|           | Points |           | Points |
|-----------|--------|-----------|--------|
| Problem 1 | 16     | Problem 4 | 17     |
| Problem 2 | 9      | Problem 5 | 18     |
| Problem 3 | 20     | Problem 6 | 18     |
| Total 98  |        |           |        |

|            |  |
|------------|--|
| First name |  |
| Last Name  |  |
| Student ID |  |

**Instructions:**

1. This is a 2-hr exam. Closed book and notes. No electronic devices or internet access.
2. A single double sided 8.5in x 11in cheat sheet is allowed.
3. If a description to an algorithm or a proof is required, please limit your description or proof to within 150 words, preferably not exceeding the space allotted for that question.
4. No space other than the pages in the exam booklet will be scanned for grading.
5. If you require an additional page for a question, you can use the extra page provided within this booklet. However please indicate clearly that you are continuing the solution on the additional page.
6. Do not detach any sheets from the booklet.
7. If using a pencil to write the answers, make sure you apply enough pressure, so your answers are readable in the scanned copy of your exam.
8. Do not write your answers in cursive scripts.
9. This exam is printed double sided. Check and use the back of each page.

1) 16 pts (2 pts each)

Mark the following statements as **TRUE** or **FALSE** by circling the correct answer. No need to provide any justification.

[ **TRUE/FALSE** ]

Bellman-Ford is asymptotically faster than Dijkstra's algorithm when dealing with graphs that only have non-negative edge weights.

[ **TRUE/FALSE** ]

In a maximum flow problem in a flow network with non-zero integer capacities, there must exist at least one integer-valued maximum flow  $f$  with non-zero  $v(f)$ .

[ **TRUE/FALSE** ]

Given a feasible circulation in a network with unlimited capacities, if the flow on each edge is increased by one unit, we will get another feasible circulation.

[ **TRUE/FALSE** ]

Suppose an edge  $e$  is not saturated due to max flow  $f$  in a Flow Network. Then increasing  $e$ 's capacity will not increase the max flow value of the network.

[ **TRUE/FALSE** ]

The worst-case time complexity of any dynamic programming algorithm with  $n^3$  unique subproblems is  $\Omega(n^3)$ .

[ **TRUE/FALSE** ]

Consider the following recurrence formula

$$OPT(j) = \max_{1 \leq i \leq j} \{A[i] - B[j - i] + OPT(j - i)\}$$

where A and B are fixed input arrays and subproblems  $OPT(j)$  are defined for  $j = 1, \dots, n$ . Then, this will lead to an  $O(n)$  dynamic programming algorithm.

[ **TRUE/FALSE** ]

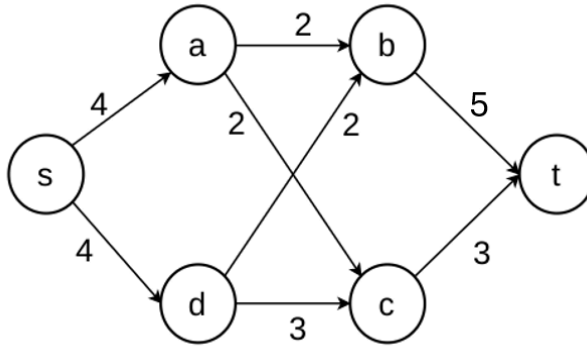
By definition, all dynamic programming algorithms must run in polynomial time with respect to the input size (either in terms of the number of integers, or in terms of the number of bits in the input).

[ **TRUE/FALSE** ]

Given a flow network  $G$  and its max flow  $f$ , we can always find an s-t cut  $(A, B)$  in  $G$  where all edges  $e$  crossing the cut either have  $f(e) = C(e)$  or  $f(e) = 0$ .

2) 9 pts - 3 pts each. No partial credit

I. What is the capacity of the minimum s-t cut in the following flow network?



- A) 5
- B) 6
- C) 7
- D) 8

**Solution: C**

II. Which of the following statements is/are False regarding dynamic programming?  
Select all correct answers!

- A) It solves problems by breaking them into overlapping subproblems.
- B) It can be implemented using a bottom-up approach known as tabulation.
- C) A problem of size  $n$  must have subproblems of size  $n/b$  where  $b > 1$  is an integer constant
- D) It can utilize memoization to store values of optimal solutions for unique subproblems to avoid redundant calculations.

**Solution: C**

III. Which of the following statements is/are True about the Ford-Fulkerson algorithm?  
Select all correct answers!

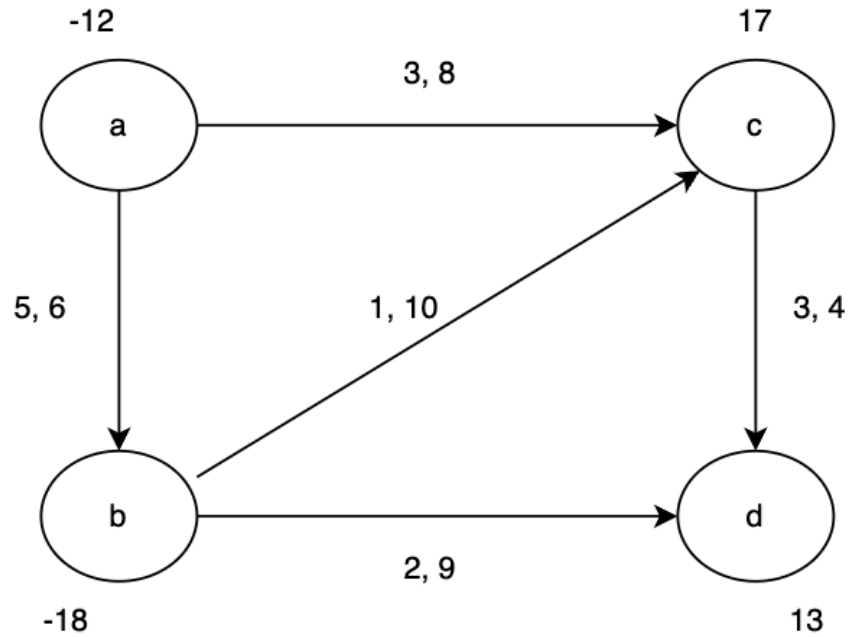
- A) For integer edge capacities, it terminates in at most  $v(f)$  iterations, where  $v(f)$  is the maximum flow value.
- B) For integer edge capacities, it terminates in at most  $C$  iterations, where  $C$  is the total capacity of edges out of source.
- C) It may converge to an incorrect value of max flow for non-integer edge capacities.
- D) Assuming it terminates, it will always find the same maximum flow  $f$  independent of the choice of augmenting paths.

**Solution: A,B,C**

3) 20 pts

Consider the flow network below with demands and lower bounds.

Notation:  $(l_e, C_e)$  show lower bound and capacity on each edge. Numbers assigned to nodes show the demand value at that node.



Follow the steps as described in class to determine if a feasible circulation exists

- a) Convert the given network to an equivalent one with no lower bounds. Show all your work. (6 pts)

Student ID:

b) Convert the network obtained in a) to an equivalent one with no demands (i.e., a flow network) (2 pts)

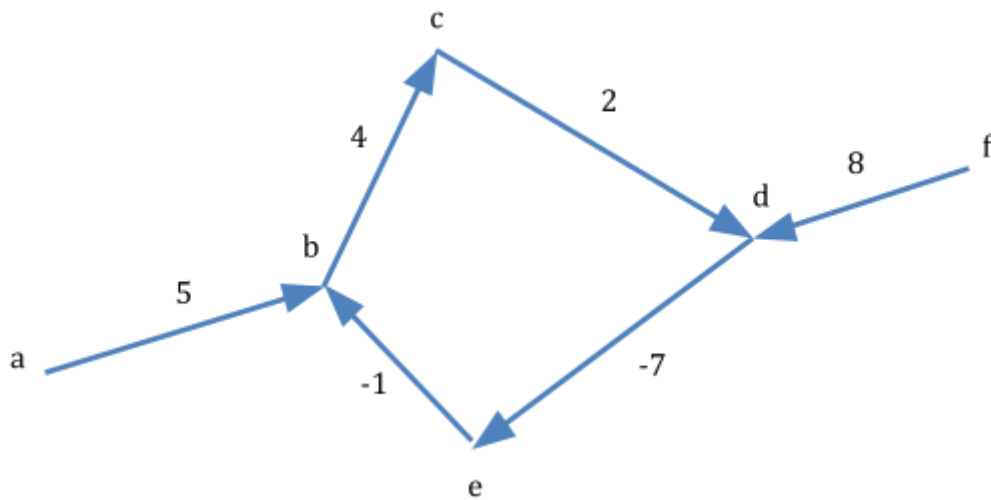
c) In the network obtained in b), compute max flow using Scaled version of Ford Fulkerson algorithm. Only show the different scaling iterations marked with the corresponding Delta value, and for each scaling iteration, show the augmenting path(s) found with the respective augmenting flow value. No need to show residual graphs. (10 pts)

Student ID:

d) Based on your solution in part c, explain whether the originally given network has a feasible circulation or not. (2 pts)

4) 17 pts

Consider the directed graph below.



- a) Use the Bellman-Ford algorithm to find the negative cycle in this graph. In other words, you must solve this problem numerically with the given edge weights and Explain how the destination node  $t$  is determined (3 pts)
- b) State how many iterations of Bellman-Ford are required (2 pts)

Student ID:

c) Show each iteration of Bellman-Ford by showing subproblem values in a table with columns representing nodes and rows representing iterations (6 pts)

d) Show how it is determined that a negative cycle exists using your results (2 pts)

e) Show how the negative cycle is found using your calculated results (4 pts)



Student ID:

5) 18 pts

Suppose USC wants to send  $m$  teams of students into a coding competition, each team consisting of one freshman and one sophomore (a student cannot be on multiple teams). Selected students are a set of freshmen  $\{s_1 \dots s_m\}$  and sophomores  $\{s_{m+1} \dots s_{2m}\}$  who need to be matched to form teams. Further, each team formed needs to be assigned a professor to mentor them. For this, a set of professors  $\{p_1 \dots p_n\}$  are available, but to ensure some load-balancing, each professor can mentor up to 5 teams each and must mentor at least one team. Each student  $s_i$  ( $i = 1, \dots, 2m$ ) is asked for a subset of professors  $X_i$  they'd like to be mentored by, and a professor  $p_k$  can be assigned to mentor a team  $(s_i, s_j)$  if  $p_k$  is in both  $X_i$  and  $X_j$ .

- a) Design a network flow algorithm to determine if it is feasible to form  $m$  teams and assign mentors given all the constraints above. (12 points)

Student ID:

b) Prove correctness of your algorithm. (6 points)

6) 18 pts

A film screening festival is being hosted in your neighborhood which will last for  $D$  days, with one movie screened each day. The list of movies consists of English, French and Korean movies.

The movie screened on day  $d$

- is in language  $L_d \in \{\text{En, Fr, Ko}\}$ ,
- has a runtime of  $t_d$  minutes (integer) and
- has a rating of  $R_d$  (may not be integer).

Given your busy schedule, you have decided you can spend at most  $T$  time for the film festival (in minutes, integer). You want to watch the best movies as much as possible, so you set the objective of maximizing the total rating of movies you watch. Your only other constraint is that you do NOT want to watch two movies in the same language back to back. For instance, if you watch a movie on day 10 and day 14 (skipping days 11-13), then  $L_{10}$  and  $L_{14}$  must be different languages.

Use dynamic programming to compute the maximum total rating of movies you can watch given the constraints above.

a) Define (in plain English) subproblems to be solved. (4 pts)

b) Write the recurrence relation(s) for the subproblems (6 pts)

Student ID:

c) Using the recurrence formula in part b, write pseudocode using iteration to compute the minimum number of packages to meet the objective. (6 pts)

Specify base cases and their values (3 pts)

Specify where the final answer can be found? (1 pt)

d) What is the complexity of your solution? (1 pt)

Is this an efficient solution? (1 pt)